КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА

Факультет комп'ютерних наук та кібернетики

ЛАБОРАТОРНА РОБОТА 1

Проблеми багатозначного аналізу

Субдиференціал негладкої опуклої функції та її мінімізація субградієнтним методом з кроком Поляка

Варіант 4

Виконав: студент групи ПМ

Кіщук Ярослав Ярославович

Київ – 2026

Зміст

1 Постановка задачі та математична модель 2
1.1 Постановка задачі 2
1.2 Властивості функції 2
1.3 Субдиференціал 3
1.4 Аналітичний розв'язок 4

2 Опис алгоритму 5
2.1 Субградієнтний метод 5
2.2 Крок Поляка 5
2.3 Критерій зупинки та алгоритм 6
2.4 Випадок невідомого f^* 6

3 Опис програми 6

4 Обчислювальні експерименти 7
4.1 Експеримент 1. Базовий запуск методу Поляка 7
4.2 Експеримент 2. Різні початкові точки 11
4.3 Експеримент 3. Неточно відоме f^* 11
4.4 Експеримент 4. Порівняння з іншими випробуваними методами 12

5 Висновки 13

6 Список використаних джерел 14

A.1 Текст програми 15

1 Постановка задачі та математична модель

1.1 Постановка задачі

Задано функцію $f : \mathbb{R}^2 \to \mathbb{R}$,

$$f(x_1, x_2) = 2x_1^2 + 3x_2^2 + |x_1 - 2x_2 + 4|, \quad x = (x_1, x_2) \in \mathbb{R}^2. \tag{1}$$

Потрібно:

1. побудувати графік субдиференціалу ∂f;
2. розв'язати задачу безумовної мінімізації

$$f(x) \to \min, \quad x \in \mathbb{R}^2, \tag{2}$$

застосовуючи один з чисельних методів негладкої оптимізації.

Позначимо $l(x) = x_1 - 2x_2 + 4 = \langle a, x\rangle + 4$, $a = (1, -2)$. Тоді $f(x) = q(x) + |l(x)|$, де $q(x) = 2x_1^2 + 3x_2^2$ — гладка частина.

1.2 Властивості функції

1. **Опуклість.** q — сильно опукла квадратична функція з гесіаном $\nabla^2 q = \operatorname{diag}(4, 6)$, а $|l(x)|$ — опукла як композиція опуклої функції $|\cdot|$ з афінною. Отже, f сильно опукла з параметром $\mu = 4$:

$$f(y) \geq f(x) + \langle g, y - x\rangle + \frac{\mu}{2}\|y - x\|^2 \quad \forall g \in \partial f(x). \tag{3}$$

2. **Існування та єдиність мінімуму.** f неперервна і $f(x) \geq 2\|x\|^2 \to \infty$ при $\|x\| \to \infty$, тому мінімум існує; з сильної опуклості він єдиний.
3. **Негладкість.** f диференційовна всюди, крім прямої

$$L = \{x \in \mathbb{R}^2 : x_1 - 2x_2 + 4 = 0\}, \tag{4}$$

де графік функції має «злам» (рис. 1).

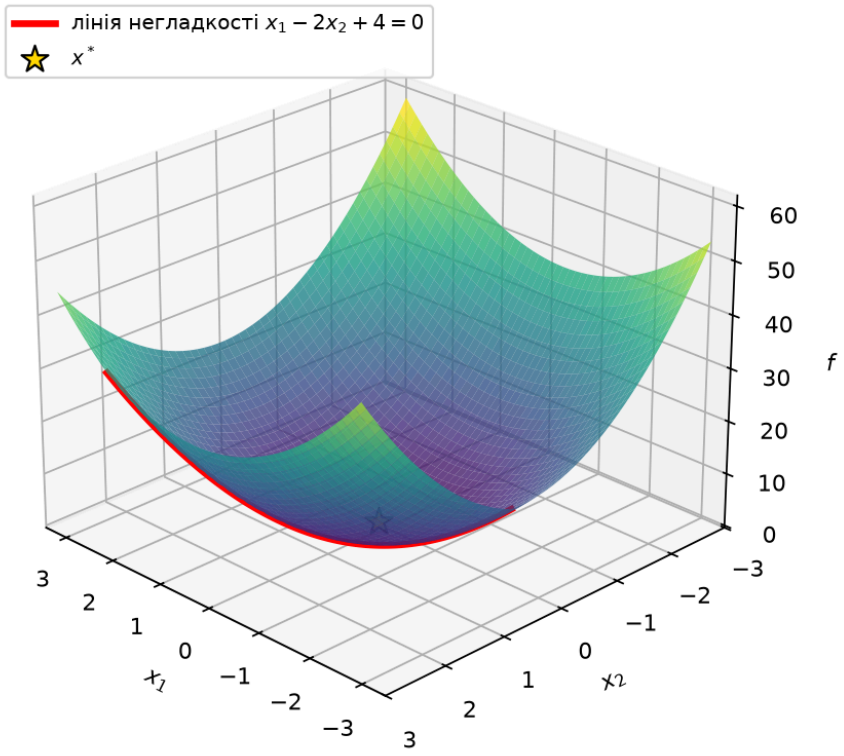


Рис. 1: Поверхня $z = f(x_1, x_2)$; червоним — лінія негладкості L, зірка — точка мінімуму x^*

1.3 Субдиференціал

Субдиференціалом опуклої функції f у точці x називається множина

$$\partial f(x) = \{g \in \mathbb{R}^2 : f(y) \geq f(x) + \langle g, y - x\rangle \quad \forall y \in \mathbb{R}^2\}. \tag{5}$$

Відображення $x \mapsto \partial f(x)$ є багатозначним відображенням $\mathbb{R}^2 \rightrightarrows \mathbb{R}^2$ з непорожніми опуклими компактними значеннями; воно монотонне і напівнеперервне зверху.

За теоремою Моро–Рокафеллара про субдиференціал суми (обидва доданки скінченні на $\mathbb{R}^2$) та правилом для композиції з афінним відображенням $\partial(h \circ l)(x) = a \cdot \partial h(l(x))$ отримуємо

$$\partial f(x) = \nabla q(x) + a \cdot \partial \mid \cdot \mid (l(x)) = \begin{pmatrix} 4x_1 \\ 6x_2 \end{pmatrix} + v \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \tag{6}$$

$$v = \operatorname{sign} l(x), \quad l(x) \neq 0; \qquad v \in [-1, 1], \quad l(x) = 0. \tag{7}$$

Отже:

- поза прямою L субдиференціал складається з однієї точки — градієнта: $\partial f(x) = \{(4x_1 + \operatorname{sign} l(x), 6x_2 - 2 \operatorname{sign} l(x))\}$;
- у точках L субдиференціал є відрізком з кінцями $(4x_1 - 1, 6x_2 + 2)$ і $(4x_1 + 1, 6x_2 - 2)$, які є границями градієнтів з двох боків прямої L; довжина відрізка $2\|a\| = 2\sqrt{5}$.

Графік $\operatorname{gph} \partial f = \{(x, g) : g \in \partial f(x)\}$ лежить у $\mathbb{R}^4$, тому на рис. 2 показано його зображення та перерізи: (а) субградієнти у точках площини: поза L — один вектор, на L — «віяло» відрізка; (б) самі множини $\partial f(x)$ у площині (g_1, g_2) для точок L — паралельні відрізки

напрямку $a = (1, -2)$; початок координат їм не належить, тож мінімум не лежить на L; (в), (г) перерізи графіка вздовж прямих $x_2 = 1/3$ і $x_1 = -1/4$, що проходять через x^*: проєкція ∂f — неперервна зростаюча гілка з вертикальним відрізком у точці перетину з L (типовий вигляд графіка монотонного багатозначного відображення).

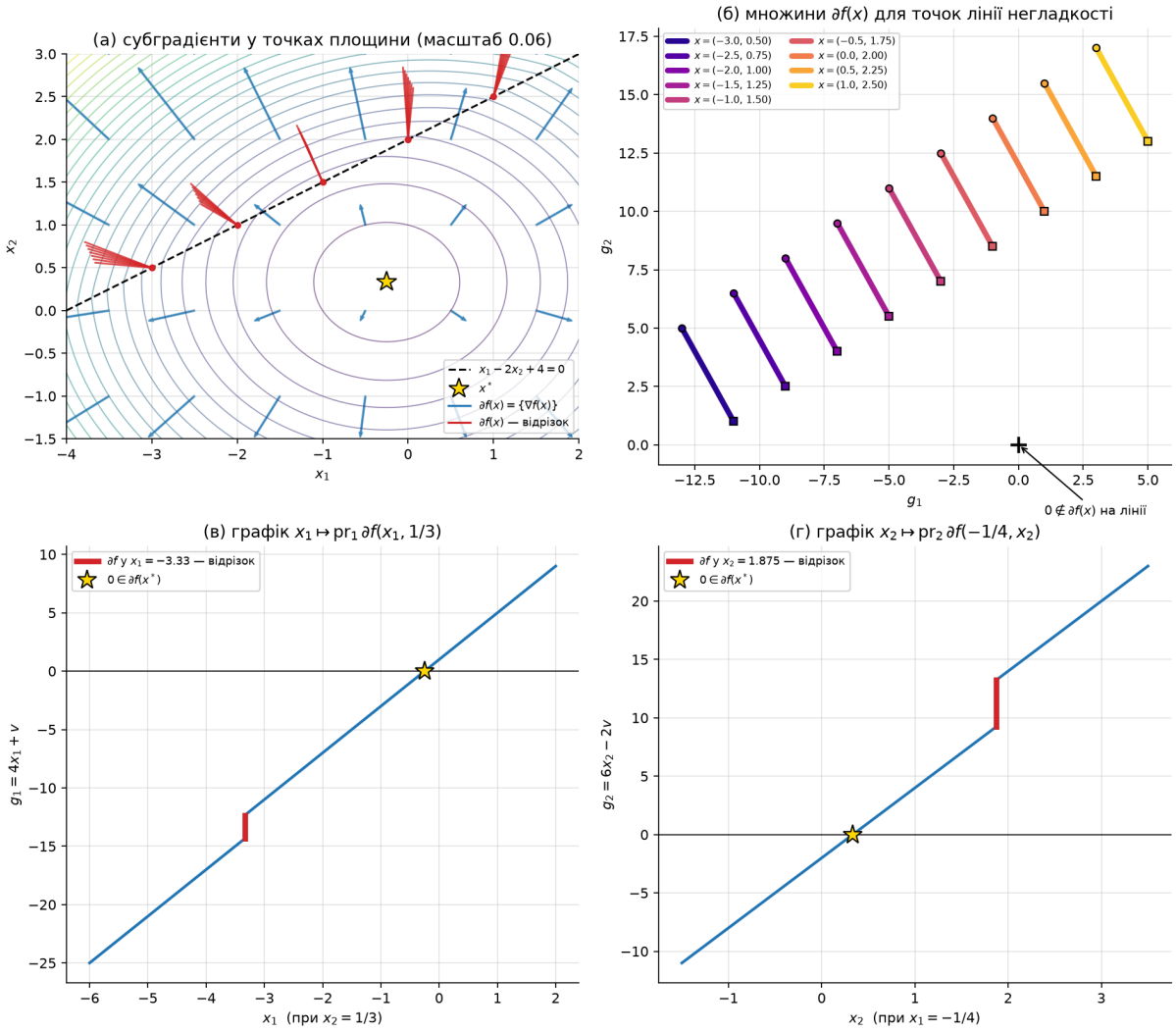


Рис. 2: Графік субдиференціалу ∂f

1.4 Аналітичний розв'язок

Для опуклої функції необхідна і достатня умова мінімуму (правило Ферма) — $0 \in \partial f(x^*)$. Розглянемо три випадки з (6).

1. $l(x) > 0$, $v = 1$: $4x_1 + 1 = 0$, $6x_2 - 2 = 0$, тобто $x = (-1/4, 1/3)$. Перевірка: $l(x) = -\frac{1}{4} - \frac{2}{3} + 4 = \frac{37}{12} > 0$ — точка підходить.
2. $l(x) < 0$, $v = -1$: $x = (1/4, -1/3)$, але $l(x) = \frac{59}{12} > 0$ — суперечність.
3. $l(x) = 0$, $v \in [-1, 1]$: $x_1 = -v/4$, $x_2 = v/3$, тоді $l(x) = 4 - 11v/12 = 0$ дає $v = 48/11 \notin [-1, 1]$ — суперечність.

Отже,

$$x^* = \left(-\frac{1}{4}, \frac{1}{3}\right), \quad f^* = f(x^*) = \frac{1}{8} + \frac{1}{3} + \frac{37}{12} = \frac{85}{24} \approx 3.5416667. \tag{8}$$

Точка мінімуму лежить в області $l(x) > 0$, де f гладка, тому поблизу x^*

$$f(x) - f^* = 2(x_1 - x_1^*)^2 + 3(x_2 - x_2^*)^2. \tag{9}$$

Відомий точний розв'язок використовується для контролю чисельного методу і для обчислення кроку Поляка.

2 Опис алгоритму

2.1 Субградієнтний метод

Субградієнтний метод (Н.З. Шор) будує послідовність

$$x^{(k+1)} = x^{(k)} - \alpha_k g^{(k)}, \quad g^{(k)} \in \partial f\big(x^{(k)}\big), \quad \alpha_k > 0. \tag{10}$$

На відміну від градієнтного методу, $-g^{(k)}$ не обов'язково є напрямком спуску, тому значення $f\big(x^{(k)}\big)$ можуть зростати, і результатом вважають рекорд $f_{\text{best}}^{(k)} = \min_{i \le k} f\big(x^{(i)}\big)$. Основою аналізу є нерівність, яка випливає з означення субградієнта, $\langle g^{(k)}, x^{(k)} - x^* \rangle \ge f\big(x^{(k)}\big) - f^*$:

$$\big\|x^{(k+1)} - x^*\big\|^2 \le \big\|x^{(k)} - x^*\big\|^2 - 2\alpha_k\big(f\big(x^{(k)}\big) - f^*\big) + \alpha_k^2 \big\|g^{(k)}\big\|^2. \tag{11}$$

2.2 Крок Поляка

Права частина (11) — квадратний тричлен відносно α_k. Його мінімум досягається при

$$\alpha_k = \frac{f\big(x^{(k)}\big) - f^*}{\big\|g^{(k)}\big\|^2}, \tag{12}$$

що і є кроком Поляка. Інша інтерпретація: α_k — довжина кроку, при якій лінійна модель $f\big(x^{(k)}\big) - \alpha\big\|g^{(k)}\big\|^2$ дорівнює f^*. Підставивши (12) у (11), отримуємо

$$\big\|x^{(k+1)} - x^*\big\|^2 \le \big\|x^{(k)} - x^*\big\|^2 - \frac{\big(f\big(x^{(k)}\big) - f^*\big)^2}{\big\|g^{(k)}\big\|^2}. \tag{13}$$

Наслідки.

1. Відстань до x^* строго спадає (фейєрівська монотонність), тож траєкторія лежить у кулі радіуса $R = \big\|x^{(0)} - x^*\big\|$, де $\|g\| \le G$.
2. Підсумовуючи (13): $\sum_k \big(f\big(x^{(k)}\big) - f^*\big)^2 \le R^2 G^2$, тобто $f\big(x^{(k)}\big) \to f^*$, і в загальному опуклому випадку точність ε гарантується за $(RG/\varepsilon)^2$ кроків.
3. **Лінійна збіжність для нашої функції.** В околі x^* (область $l(x) > 0$) функція збігається з квадратичною (9) з $\mu = 4$, $L = 6$, тому $\|g\|^2 \le 2L(f - f^*)$ і $f - f^* \ge \frac{\mu}{2}\|x - x^*\|^2$. З (13):

$$\big\|x^{(k+1)} - x^*\big\|^2 \le \left(1 - \frac{\mu}{4L}\right)\big\|x^{(k)} - x^*\big\|^2 = \frac{5}{6}\big\|x^{(k)} - x^*\big\|^2. \tag{14}$$

Фактично швидкість ще вища: якщо похибка $e = x - x^*$ спрямована вздовж власного вектора гесіана з власним числом λ, то $\alpha = (\lambda e^2/2)/(\lambda^2 e^2) = 1/(2\lambda)$ і $x^{(k+1)} - x^* = e/2$ — відстань зменшується вдвічі, а $f - f^*$ — вчетверо за ітерацію. Компонента вздовж x_2 ($\lambda = 6$) згасає швидше, тому похибка вирівнюється вздовж x_1 ($\lambda = 4$), і $\alpha_k \to 1/8$.

2.3 Критерій зупинки та алгоритм

Ітерації припиняються, якщо $f\big(x^{(k)}\big) - f^* \le \varepsilon$ ($\varepsilon = 10^{-14}$, рівень машинної точності для $f \approx 3.5$), або $0 \in \partial f\big(x^{(k)}\big)$, або $k = k_{\max}$.

На прямій L як субградієнт береться елемент відрізка $\partial f(x)$ з найменшою нормою — проєкція нуля на відрізок; за правилом Ферма, якщо він нульовий, то x — розв'язок.

Алгоритм.

1. Задати $x^{(0)}$, f^*, ε, $k_{\max}$; покласти $k = 0$.
2. Обчислити $f\big(x^{(k)}\big)$. Якщо $f\big(x^{(k)}\big) - f^* \le \varepsilon$ або $k = k_{\max}$ — зупинитися.
3. Обчислити $g^{(k)} \in \partial f\big(x^{(k)}\big)$ за (6). Якщо $g^{(k)} = 0$ — зупинитися.
4. Обчислити α_k за (12) і $x^{(k+1)} = x^{(k)} - \alpha_k g^{(k)}$.
5. $k := k + 1$, перейти до кроку 2.

2.4 Випадок невідомого f^*

У нашій задачі f^* відомо аналітично, але метод можна застосувати й без нього, замінивши f^* у (12) оцінкою $f_{\text{lev}}^{(k)}$:

- **оцінка за рекордом** [4, §4.2]: $f_{\text{lev}}^{(k)} = f_{\text{best}}^{(k)} - \gamma_k$, $\gamma_k = 10/(10 + k)$, $\gamma_k \to 0$, $\sum \gamma_k = \infty$;
- **динамічний цільовий рівень** (у дусі [5]): $f_{\text{lev}}^{(k)} = f_{\text{best}}^{(k)} - \delta_k$, де $\delta_{k+1} = 1.5\delta_k$, якщо досягнуто достатнього спуску $f\big(x^{(k+1)}\big) \le f_{\text{best}}^{(k)} - \delta_k/2$, і $\delta_{k+1} = \max(\delta_k/2, 10^{-12})$ інакше.

Обидва варіанти досліджено в експерименті 3.

3 Опис програми

Програму написано мовою Python 3 (NumPy, Matplotlib); вона міститься у файлі `nonsmooth_optimization_solvers.py` (повний текст — у додатку А) і запускається командою `python nonsmooth_optimization_solvers.py`. Ноутбук `lab1.ipynb` виконує той самий скрипт і показує результати. Основні функції наведено в табл. 1.

Табл. 1: Основні функції програми

Функція	**Призначення**
`f(x)`, `kink(x)`	цільова функція (1) та $l(x)$
`subdifferential(x)`	кінці відрізка $\partial f(x)$ за (6) (поза L — одна точка)
`subgradient(x)`	субградієнт: градієнт поза L, елемент з мінімальною нормою на L
`polyak_method(x0, f_target, eps, max_iter)`	основний метод — крок Поляка (12)
`polyak_estimated_fstar`, `polyak_target_level`	варіанти для невідомого f^* (п. 2.4)
`diminishing_step_method`, `constant_step_method`, `momentum_method`	інші випробувані методи (для порівняння, п. 4.4)
`plot_surface`, `plot_subdifferential`	рис. 1, рис. 2
`make_polyak_animation`, `make_compare_animation`	анімації GIF/MP4 і кадри для звіту
`main()`	проведення експериментів, вивід таблиць, збереження `results.json` і рисунків

Кожен метод повертає словник з історією точок $x^{(k)}$, значень $f\big(x^{(k)}\big)$, рекордів $f_{\text{best}}^{(k)}$, кроків α_k і найкращої точки. Числові результати записуються у `report/figures/results.json`, звідки їх безпосередньо читає цей звіт, тож таблиці відповідають фактичному запуску програми. Додатково програма створює анімації `polyak_anim.gif`/`.mp4` (робота методу Поляка) та `methods_compare.gif`/`.mp4` (одночасний рух усіх методів), які додаються до звіту окремими файлами.

4 Обчислювальні експерименти

Точний розв'язок (8): $x^* = (-0.25, 0.333333)$, $f^* = 3.5416667$. Якщо не вказано інше, $x^{(0)} = (-4, 3)$: ця точка лежить в області $l(x) < 0$, тож траєкторія перетинає лінію негладкості.

4.1 Експеримент 1. Базовий запуск методу Поляка

Метод зупинився через 27 ітерацій з $f\big(x^{(k)}\big) - f^* = 4.00 \cdot 10^{-15}$ та $\big\|x^{(k)} - x^*\big\| = 4.42 \cdot 10^{-8}$. Перші ітерації наведено в табл. 2.

Табл. 2: Ітерації методу Поляка, $x^{(0)} = (-4, 3)$ (перші 12 і останні 3)

k	$x_1^{(k)}$	$x_2^{(k)}$	$f\big(x^{(k)}\big)$	$l\big(x^{(k)}\big)$	α_k	$\big\|x^{(k)} - x^*\big\|$
0	−4.000000	3.000000	65.00000000	−6.0000	0.08920	4.60
1	−2.483612	1.216014	17.68835775	−0.9156	0.06868	2.40
2	−1.732632	0.577557	8.11699635	1.1123	0.12260	1.50
3	−1.005534	0.397902	4.69583694	2.1987	0.12433	$7.58 \cdot 10^{-1}$
4	−0.629803	0.349737	3.83097417	2.6707	0.12483	$3.80 \cdot 10^{-1}$
5	−0.440166	0.337451	3.61404371	2.8849	0.12496	$1.90 \cdot 10^{-1}$
6	−0.345116	0.334364	3.55976410	2.9862	0.12499	$9.51 \cdot 10^{-2}$
7	−0.297562	0.333591	3.54619122	3.0353	0.12500	$4.76 \cdot 10^{-2}$
8	−0.273782	0.333398	3.54279782	3.0594	0.12500	$2.38 \cdot 10^{-2}$
9	−0.261891	0.333349	3.54194946	3.0714	0.12500	$1.19 \cdot 10^{-2}$
10	−0.255945	0.333337	3.54173736	3.0774	0.12500	$5.95 \cdot 10^{-3}$
11	−0.252973	0.333334	3.54168434	3.0804	0.12500	$2.97 \cdot 10^{-3}$
25	−0.25000018	0.33333333	3.5416666667	3.0833	0.12596	$1.81 \cdot 10^{-7}$
26	−0.25000009	0.33333333	3.5416666667	3.0833	0.12707	$8.99 \cdot 10^{-8}$
27	−0.25000004	0.33333333	3.5416666667	3.0833	—	$4.42 \cdot 10^{-8}$

На першому кроці точка переходить в область $l(x) > 0$ майже до лінії L, на другому перетинає L, після чого рухається в гладкій області. Крок α_k швидко стабілізується на $1/8 = 0.125$, а відстань до x^* зменшується рівно вдвічі за ітерацію, як і передбачає аналіз у п. 2.2 (рис. 3, рис. 4).

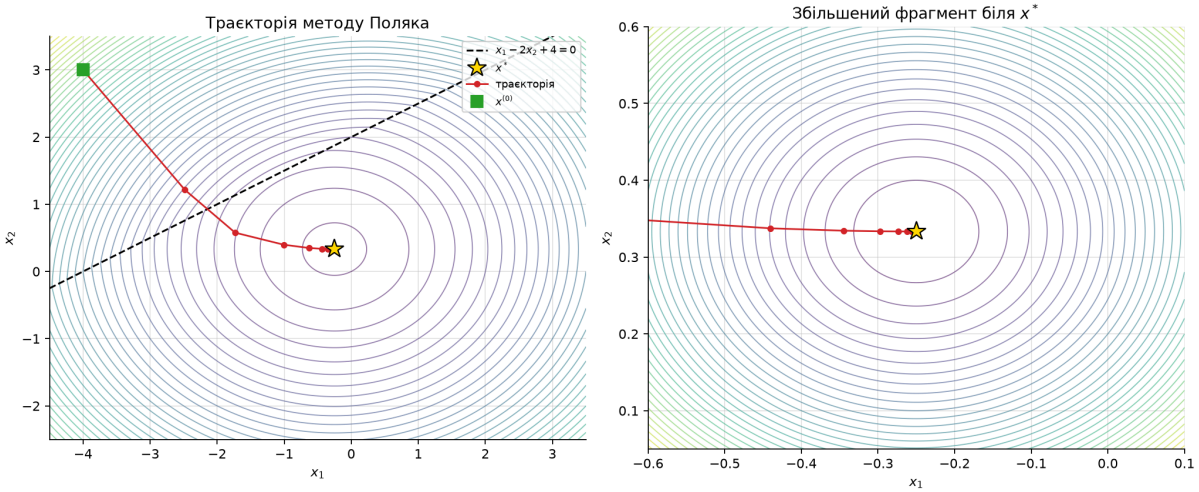


Рис. 3: Траєкторія методу Поляка на лініях рівня f (пунктир — лінія негладкості L) і її збільшений фрагмент біля x^*

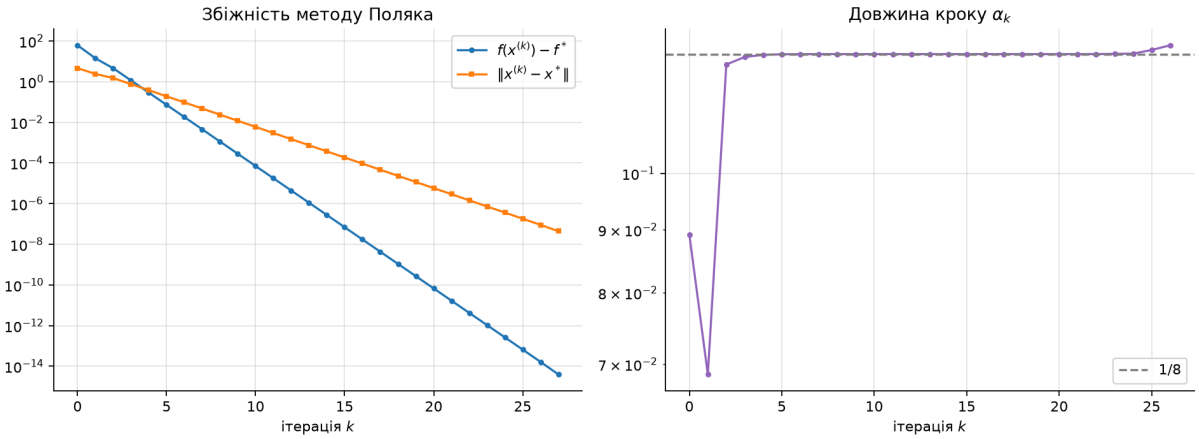


Рис. 4: Ліворуч: $f\left(x^{(k)}\right) - f^*$ і $\left\|x^{(k)} - x^*\right\|$ (логарифмічна шкала); праворуч: довжина кроку α_k

Динаміку методу зручно спостерігати на анімації `polyak_anim.mp4` (`polyak_anim.gif`), що додається до звіту. На рис. 5 — її ключові кадри: ліворуч траєкторія і напрям поточного кроку, посередині «камера» у координатах $x - x^*$, вікно якої зменшується разом з відстанню до розв'язку (від 10^0 до 10^{-8}), праворуч графік $f - f^*$.

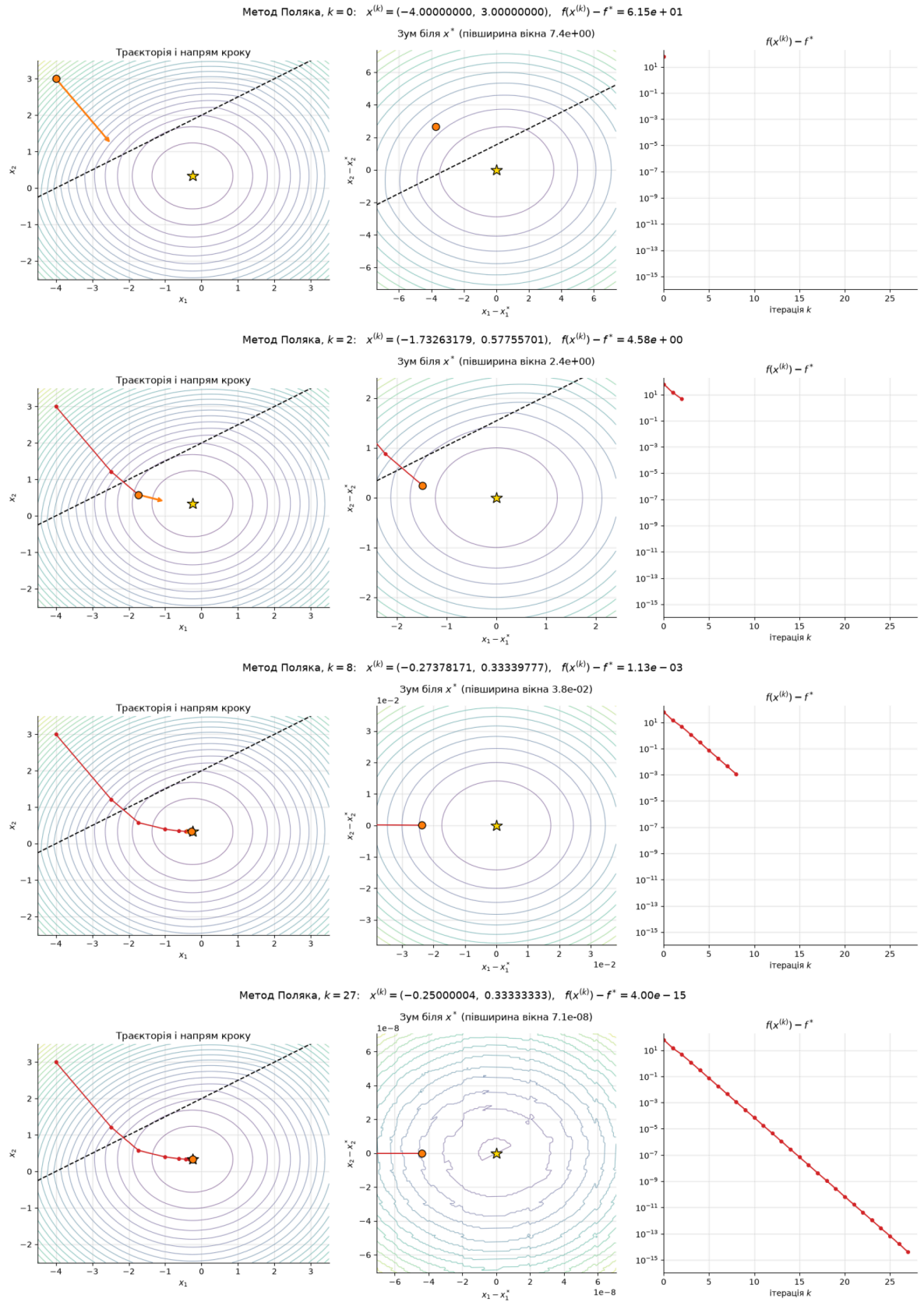


Рис. 5: Кадри анімації методу Поляка ($k = 0, 2, 8$ і останній)

4.2 Експеримент 2. Різні початкові точки

Розглянуто точки з обох боків прямої L і дві точки на самій L ($l(x^{(0)}) = 0$), у яких субдиференціал є відрізком (табл. 3, рис. 6).

Табл. 3: Метод Поляка з різних початкових точок ($\varepsilon = 10^{-14}$)

$x^{(0)}$	$l(x^{(0)})$	ітерацій	перетинів L	$f_{\text{best}} - f^*$	$\|x_{\text{best}} - x^*\|$
$(3, -2)$	11.0	26	0	$9.33 \cdot 10^{-15}$	$6.76 \cdot 10^{-8}$
$(-4, 3)$	-6.0	27	1	$4.00 \cdot 10^{-15}$	$4.42 \cdot 10^{-8}$
$(10, 10)$	-6.0	28	1	$7.99 \cdot 10^{-15}$	$6.27 \cdot 10^{-8}$
$(-2, 1)$	0.0	25	1	$6.66 \cdot 10^{-15}$	$5.81 \cdot 10^{-8}$
$(-5, -3)$	5.0	27	0	$4.88 \cdot 10^{-15}$	$4.91 \cdot 10^{-8}$
$(0, 2)$	0.0	25	1	$9.33 \cdot 10^{-15}$	$6.78 \cdot 10^{-8}$

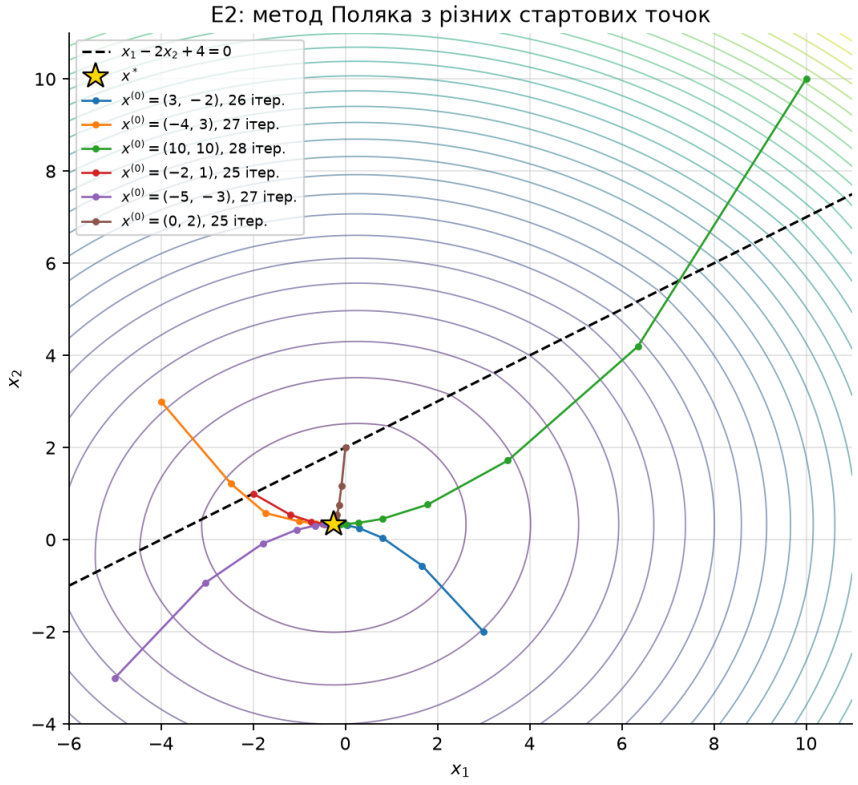


Рис. 6: Траєкторії методу Поляка з різних початкових точок

Кількість ітерацій практично не залежить від початкової точки (25–28): після одного-двох великих кроків траєкторія потрапляє в гладку область біля x^*, де збіжність лінійна зі знаменником $1/2$; віддаленість точки впливає лише логарифмічно. Старт на лінії негладкості не створює труднощів завдяки вибору субградієнта з мінімальною нормою.

4.3 Експеримент 3. Неточно відоме f^*

Крок Поляка вимагає значення f^*. Щоб оцінити чутливість методу, у (12) замість f^* підставлено $f^* + \delta$; кількість ітерацій обмежено $k_{\max} = 200$.

Табл. 4: Крок Поляка з неточним цільовим рівнем, $x^{(0)} = (-4, 3)$

δ	ітерацій	$f_{\text{best}} - f^*$	$\|x_{\text{best}} - x^*\|$	поведінка
$1 \cdot 10^{-1}$	9	$1.00 \cdot 10^{-1}$	$2.24 \cdot 10^{-1}$	зупинка на рівні $f^* + \delta$
$1 \cdot 10^{-2}$	11	$1.00 \cdot 10^{-2}$	$7.07 \cdot 10^{-2}$	зупинка на рівні $f^* + \delta$
$1 \cdot 10^{-3}$	12	$1.00 \cdot 10^{-3}$	$2.24 \cdot 10^{-2}$	зупинка на рівні $f^* + \delta$
$-1 \cdot 10^{-3}$	200	$3.00 \cdot 10^{-7}$	$3.87 \cdot 10^{-4}$	коливання біля x^*
$-1 \cdot 10^{-2}$	200	$6.68 \cdot 10^{-6}$	$1.81 \cdot 10^{-3}$	коливання біля x^*
$-1 \cdot 10^{-1}$	200	$5.82 \cdot 10^{-5}$	$5.23 \cdot 10^{-3}$	коливання біля x^*
—	200	$3.15 \cdot 10^{1}$	3.34	$f_{\text{best}} - 10/(10 + k)$
—	200	0	$4.01 \cdot 10^{-9}$	динамічний рівень

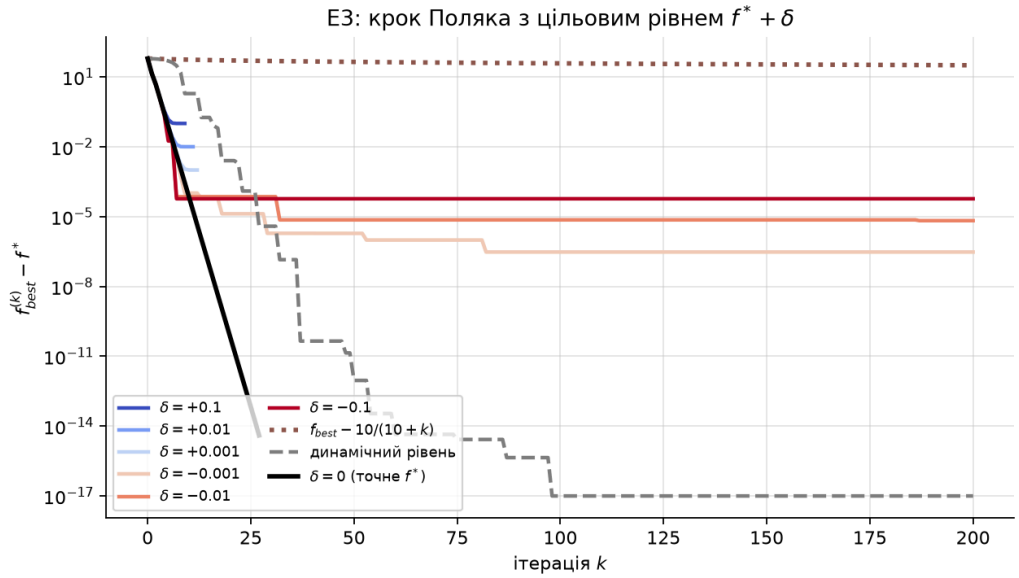


Рис. 7: Збіжність $f_{\text{best}}^{(k)} - f^*$ при різних цільових рівнях

Результати (табл. 4, рис. 7):

- **переоцінка** ($\delta > 0$): метод швидко досягає рівня $f^* + \delta$ і зупиняється, бо крок стає нульовим; точність обмежена величиною δ;
- **недооцінка** ($\delta < 0$): кроки завеликі, метод не зупиняється й коливається біля x^*, проте завдяки сильній опуклості рекорд f_{best} все одно значно кращий за $|\delta|$;
- оцінка $f_{\text{best}} - \gamma_k$ з [4] теоретично збігається, але на цій функції дуже повільно: крок $\gamma_k / \|g\|^2$ малий, бо $\|g\|$ великий далеко від мінімуму;
- динамічний рівень без знання f^* досягає машинної точності приблизно за 100 ітерацій — повільніше, ніж з точним f^* (27 ітерацій), але значно швидше за методи з наперед заданими кроками.

4.4 Експеримент 4. Порівняння з іншими випробуваними методами

Окрім кроку Поляка, було випробувано три інші правила вибору кроку в (10) (з нормованим напрямком $g/\|g\|$):

- спадний крок $h_k = 0.6/\sqrt{k+1}$;
- сталий крок $h_k = 0.05$;
- усереднення напрямків (momentum): $d^{(k)} = 0.7d^{(k-1)} + 0.3g^{(k)}$, $x^{(k+1)} = x^{(k)} - h_k d^{(k)}/\|d^{(k)}\|$, $h_k = 0.6/\sqrt{k+1}$.

Усі методи стартували з $x^{(0)} = (-4, 3)$ з бюджетом 200 ітерацій (табл. 5, рис. 8; анімація `methods_compare.mp4`).

Табл. 5: Порівняння методів, $x^{(0)} = (-4, 3)$

Метод	ітерацій	k: $f_{\text{best}} - f^* < 10^{-3}$	$f_{\text{best}} - f^*$	$\|x_{\text{best}} - x^*\|$
Крок Поляка	27	9	$4.00 \cdot 10^{-15}$	$4.42 \cdot 10^{-8}$
Спадний крок	200	121	$4.49 \cdot 10^{-4}$	$1.22 \cdot 10^{-2}$
Сталий крок	200	95	$4.62 \cdot 10^{-4}$	$1.52 \cdot 10^{-2}$
Усереднення (momentum)	200	28	$2.73 \cdot 10^{-5}$	$3.02 \cdot 10^{-3}$

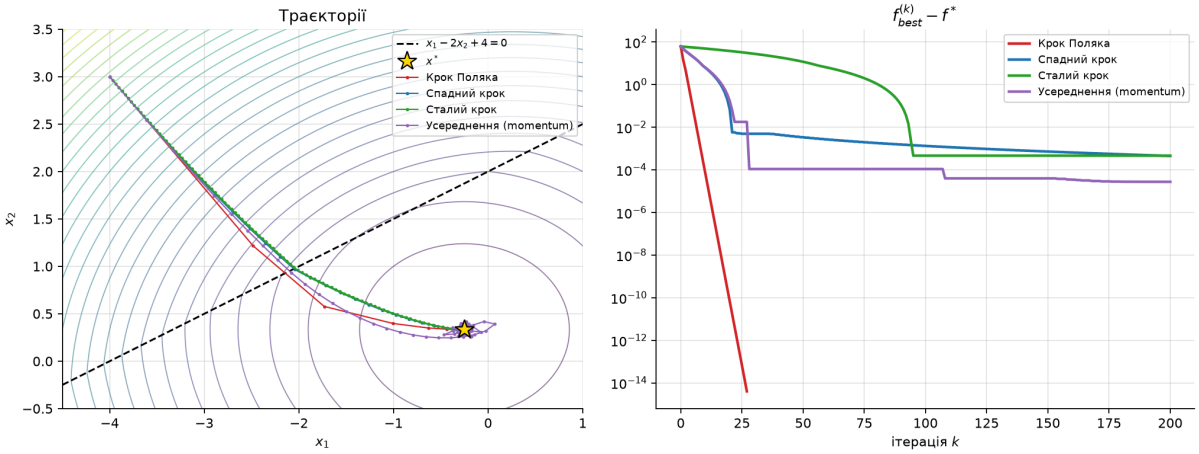


Рис. 8: Траєкторії методів і збіжність $f_{\text{best}}^{(k)} - f^*$

Методи з наперед заданим кроком не використовують інформацію про значення функції: крок не залежить від відстані до розв'язку, тому біля x^* вони «перестрибують» мінімум і коливаються з амплітудою порядку h_k. Збіжність у них сублінійна ($O\left(1/\sqrt{k}\right)$), і за 200 ітерацій досягнуто лише 10^{-4}–10^{-5} за функцією. Крок Поляка автоматично масштабується пропорційно $f\left(x^{(k)}\right) - f^*$ і на цій сильно опуклій функції дає лінійну збіжність, досягаючи машинної точності за 27 ітерацій. Тому саме його обрано основним методом.

5 Висновки

1. Для функції $f(x) = 2x_1^2 + 3x_2^2 + |x_1 - 2x_2 + 4|$ знайдено субдиференціал: поза прямою $L : x_1 - 2x_2 + 4 = 0$ він збігається з градієнтом, а на L є відрізком довжини $2\sqrt{5}$ з напрямком $(1, -2)$. Побудовано графік субдиференціалу (зображення в площині, множини $\partial f(x)$ і перерізи графіка).
2. З умови $0 \in \partial f(x^*)$ знайдено точний розв'язок $x^* = (-1/4, 1/3)$, $f^* = 85/24$; мінімум лежить поза лінією негладкості.

3. Реалізовано субградієнтний метод з кроком Поляка. Він збігається з довільної початкової точки (у тому числі з точок на L) за 25–28 ітерацій до $\|x - x^*\| \sim 10^{-8}$, $f - f^* \sim 10^{-15}$; спостережувану лінійну збіжність зі знаменником $1/2$ пояснено теоретично.
4. Метод чутливий до значення f^*: переоцінка обмежує точність, недооцінка спричиняє коливання. За невідомого f^* ефективним виявився динамічний цільовий рівень.
5. Порівняння з іншими випробуваними правилами кроку (спадний, сталий, усереднення напрямків) показало, що вони за 200 ітерацій дають точність лише 10^{-4}–10^{-5}, тоді як крок Поляка — машинну точність за 27 ітерацій.

6 Список використаних джерел

1. Поляк Б.Т. Введение в оптимизацию. — М.: Наука, 1983. — 384 с.
2. Шор Н.З. Методы минимизации недифференцируемых функций и их приложения. — К.: Наукова думка, 1979. — 200 с.
3. Рокафеллар Р. Выпуклый анализ. — М.: Мир, 1973. — 469 с.
4. Boyd S., Park J. Subgradient Methods: Notes for EE364b. — Stanford University, 2014. — 39 p.
5. Nedić A., Bertsekas D.P. Incremental subgradient methods for nondifferentiable optimization
6. Жалдак М.І., Триус Ю.В. Основи теорії і методів оптимізації: навч. посібник. — Черкаси: Брама-Україна, 2005. — 608 с.

A.1 Текст програми

Файл `nonsmooth_optimization_solvers.py`.

```
"""
Лабораторна робота 1 з курсу «Проблеми багатозначного аналізу», варіант 4.

Задача:
    f(x1, x2) = 2*x1^2 + 3*x2^2 + |x1 - 2*x2 + 4|  ->  min,  x = (x1, x2) ∈ R^2.

Точний розв'язок (з умови 0 ∈ ∂f(x*)):
    x* = (-1/4, 1/3),  f(x*) = 85/24 ≈ 3.54166667.

Основний метод — субградієнтний метод з кроком Поляка. Для порівняння реалізовано
ще три варіанти субградієнтного методу (спадний, сталий крок, усереднення напрямків).

Скрипт:
  1. розв'язує задачу і проводить обчислювальні експерименти;
  2. зберігає числові результати у report/figures/results.json (їх читає звіт);
  3. будує рисунки (поверхня, субдиференціал, траєкторії, збіжність) у report/figures/;
  4. створює анімації роботи методів у форматах GIF і MP4.

Запуск:  python nonsmooth_optimization_solvers.py
"""

import json
import os

import matplotlib

matplotlib.use("Agg")  # рисунки лише зберігаються у файли, без вікон
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FFMpegWriter, FuncAnimation, PillowWriter

FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "report", "figures")

# ---------------------------------------------------------------------------
# 1. Цільова функція, субградієнт і субдиференціал
# ---------------------------------------------------------------------------
A_VEC = np.array([1.0, -2.0])  # нормаль до лінії негладкості x1 - 2*x2 + 4 = 0
B_CONST = 4.0


def kink(x):
    """Значення лінійної функції під модулем: l(x) = x1 - 2*x2 + 4."""
    return x[0] - 2 * x[1] + B_CONST


def f(x):
    """Цільова функція f(x1, x2) = 2*x1^2 + 3*x2^2 + |x1 - 2*x2 + 4|."""
    return 2 * x[0] ** 2 + 3 * x[1] ** 2 + np.abs(kink(x))


def grad_smooth(x):
    """Градієнт гладкої частини 2*x1^2 + 3*x2^2."""
    return np.array([4 * x[0], 6 * x[1]])


def subdifferential(x, tol=1e-12):
    """
    Субдиференціал ∂f(x) = (4*x1, 6*x2) + v*(1, -2), де
        v = sign(l(x)),  якщо l(x) != 0,
        v ∈ [-1, 1],     якщо l(x) = 0  (точка на лінії негладкості).
    Повертає кінці відрізка (p, q): при l(x) != 0 відрізок вироджується в точку p = q.
    """
    g0 = grad_smooth(x)
    l = kink(x)
    if abs(l) > tol:
```

```
        p = g0 + np.sign(l) * A_VEC
        return p, p.copy()
    return g0 - A_VEC, g0 + A_VEC


def subgradient(x, tol=1e-12):
    """
    Один субградієнт g ∈ ∂f(x). Поза лінією негладкості це градієнт.
    На лінії l(x) = 0 беремо елемент з мінімальною нормою відрізка ∂f(x)
    (він теж належить ∂f(x), а якщо він нульовий — x є точкою мінімуму).
    """
    p, q = subdifferential(x, tol)
    d = q - p
    if not np.any(d):
        return p
    t = np.clip(-(p @ d) / (d @ d), 0.0, 1.0)  # проєкція нуля на відрізок [p, q]
    return p + t * d


# Точний розв'язок: x* лежить в області l(x) > 0, де f гладка,
# тому 0 = (4*x1 + 1, 6*x2 - 2)  =>  x* = (-1/4, 1/3).
X_STAR = np.array([-0.25, 1.0 / 3.0])
F_STAR = 85.0 / 24.0
assert abs(f(X_STAR) - F_STAR) < 1e-14 and kink(X_STAR) > 0


# ---------------------------------------------------------------------------
# 2. Методи негладкої оптимізації
# ---------------------------------------------------------------------------
def _result(name, xs, alphas=None):
    """Пакує історію ітерацій у словник: точки, значення f, рекорди f_best."""
    xs = np.array(xs)
    fs = np.array([f(x) for x in xs])
    f_best = np.minimum.accumulate(fs)
    k_best = int(np.argmin(fs))
    return {
        "name": name,
        "x": xs,
        "f": fs,
        "f_best": f_best,
        "x_best": xs[k_best],
        "n_iter": len(xs) - 1,
        "alpha": np.array(alphas if alphas is not None else []),
    }


def polyak_method(x0, f_target=F_STAR, eps=1e-14, max_iter=200):
    """
    ОСНОВНИЙ МЕТОД. Субградієнтний метод з кроком Поляка:
        x^(k+1) = x^(k) - alpha_k * g^(k),   alpha_k = (f(x^(k)) - f_target) / ||g^(k)||^2,
    де g^(k) ∈ ∂f(x^(k)), f_target — відоме (або оцінене) значення мінімуму.
    Зупинка: f(x^(k)) - f_target <= eps, або 0 ∈ ∂f(x^(k)), або k = max_iter.
    """
    x = np.array(x0, dtype=float)
    xs, alphas = [x.copy()], []
    for _ in range(max_iter):
        gap = f(x) - f_target
        if gap <= eps:  # досягнуто цільового рівня
            break
        g = subgradient(x)
        g_norm2 = g @ g
        if g_norm2 == 0.0:  # 0 ∈ ∂f(x) — точка мінімуму
            break
        alpha = gap / g_norm2
        x = x - alpha * g
        xs.append(x.copy())
        alphas.append(alpha)
    return _result("Крок Поляка", xs, alphas)
```

```
def polyak_estimated_fstar(x0, c=10.0, max_iter=200):
    """
    Крок Поляка з оцінкою f* ≈ f_best - gamma_k, gamma_k = c / (c + k) -> 0, sum gamma_k = inf
    (Boyd, Subgradient Methods, §4.2):  alpha_k = (f(x^(k)) - f_best^(k) + gamma_k) / ||g^(k)||^2.
    """
    x = np.array(x0, dtype=float)
    xs, alphas = [x.copy()], []
    f_best = f(x)
    for k in range(1, max_iter + 1):
        g = subgradient(x)
        g_norm2 = g @ g
        if g_norm2 == 0.0:
            break
        alpha = (f(x) - f_best + c / (c + k)) / g_norm2
        x = x - alpha * g
        f_best = min(f_best, f(x))
        xs.append(x.copy())
        alphas.append(alpha)
    return _result("Поляк, оцінка f_best - gamma_k", xs, alphas)


def polyak_target_level(x0, delta0=1.0, rho=1.5, beta=0.5, delta_min=1e-12, max_iter=200):
    """
    Варіант кроку Поляка для НЕВІДОМОГО f* (динамічний цільовий рівень у дусі
    Nedić–Bertsekas): замість f* береться f_lev = f_best - delta_k і
        delta_(k+1) = rho * delta_k,                  якщо f(x^(k+1)) <= f_best - delta_k / 2
                                                      (достатній спуск — рівень можна знижувати),
        delta_(k+1) = max(beta * delta_k, delta_min),  інакше (рівень занадто низький).
    """
    x = np.array(x0, dtype=float)
    xs, alphas = [x.copy()], []
    f_best, delta = f(x), delta0
    for _ in range(max_iter):
        g = subgradient(x)
        g_norm2 = g @ g
        if g_norm2 == 0.0:
            break
        f_lev = f_best - delta
        alpha = (f(x) - f_lev) / g_norm2
        x = x - alpha * g
        fx = f(x)
        delta = rho * delta if fx <= f_best - delta / 2 else max(beta * delta, delta_min)
        f_best = min(f_best, fx)
        xs.append(x.copy())
        alphas.append(alpha)
    return _result("Поляк, динамічний рівень", xs, alphas)


# --- Інші спроби (для порівняння) ------------------------------------------
def diminishing_step_method(x0, a=0.6, max_iter=200):
    """Субградієнтний метод з нормованим спадним кроком h_k = a / sqrt(k+1)."""
    x = np.array(x0, dtype=float)
    xs = [x.copy()]
    for k in range(max_iter):
        g = subgradient(x)
        norm_g = np.linalg.norm(g)
        if norm_g < 1e-12:
            break
        x = x - a / np.sqrt(k + 1) * g / norm_g
        xs.append(x.copy())
    return _result("Спадний крок", xs)


def constant_step_method(x0, a=0.05, max_iter=200):
    """Субградієнтний метод зі сталим нормованим кроком h_k = a."""
    x = np.array(x0, dtype=float)
    xs = [x.copy()]
    for _ in range(max_iter):
        g = subgradient(x)
        norm_g = np.linalg.norm(g)
```

```
        if norm_g < 1e-12:
            break
        x = x - a * g / norm_g
        xs.append(x.copy())
    return _result("Сталий крок", xs)


def momentum_method(x0, beta=0.7, a=0.6, max_iter=200):
    """
    Субградієнтний метод з усередненням напрямків (heavy ball / CFM):
        d^(k) = beta * d^(k-1) + (1 - beta) * g^(k),   x^(k+1) = x^(k) - h_k * d^(k) / ||d^(k)||.
    """
    x = np.array(x0, dtype=float)
    xs = [x.copy()]
    d = np.zeros(2)
    for k in range(max_iter):
        d = beta * d + (1 - beta) * subgradient(x)
        norm_d = np.linalg.norm(d)
        if norm_d < 1e-12:
            break
        x = x - a / np.sqrt(k + 1) * d / norm_d
        xs.append(x.copy())
    return _result("Усереднення (momentum)", xs)


# ---------------------------------------------------------------------------
# 3. Допоміжні функції для графіків
# ---------------------------------------------------------------------------
COLORS = {
    "Крок Поляка": "#d62728",
    "Спадний крок": "#1f77b4",
    "Сталий крок": "#2ca02c",
    "Усереднення (momentum)": "#9467bd",
}

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "axes.grid": True,
    "grid.alpha": 0.3,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "figure.dpi": 100,
})


def contour_background(ax, xlim, ylim, levels=25):
    """Лінії рівня f і лінія негладкості x1 - 2*x2 + 4 = 0."""
    X1, X2 = np.meshgrid(np.linspace(*xlim, 400), np.linspace(*ylim, 400))
    Z = f(np.array([X1, X2]))
    ax.contour(X1, X2, Z, levels=levels, cmap="viridis", alpha=0.55, linewidths=0.8)
    t = np.linspace(*xlim, 10)
    ax.plot(t, (t + B_CONST) / 2, "k--", lw=1.4, label=r"$x_1 - 2x_2 + 4 = 0$")
    ax.plot(*X_STAR, "*", color="gold", mec="k", ms=15, zorder=5, label=r"$x^*$")
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    ax.set_xlabel(r"$x_1$")
    ax.set_ylabel(r"$x_2$")
    ax.set_aspect("equal")


def save(fig, name):
    path = os.path.join(FIG_DIR, name)
    fig.savefig(path, dpi=160, bbox_inches="tight")
    plt.close(fig)
    print("  збережено", os.path.relpath(path))


def plot_surface():
    """3D-поверхня f з лінією негладкості та точкою мінімуму."""
    fig = plt.figure(figsize=(8, 6))
```

```
    ax = fig.add_subplot(111, projection="3d")
    X1, X2 = np.meshgrid(np.linspace(-3, 3, 150), np.linspace(-3, 3, 150))
    Z = f(np.array([X1, X2]))
    ax.plot_surface(X1, X2, Z, cmap="viridis", alpha=0.8, edgecolor="none")
    t = np.linspace(-3, 2, 100)
    ax.plot(t, (t + 4) / 2, 2 * t ** 2 + 3 * ((t + 4) / 2) ** 2, "r-", lw=3,
            label=r"лінія негладкості $x_1 - 2x_2 + 4 = 0$")
    ax.scatter(*X_STAR, F_STAR, color="gold", edgecolor="k", s=150, marker="*", label=r"$x^*$")
    ax.set_xlabel(r"$x_1$")
    ax.set_ylabel(r"$x_2$")
    ax.set_zlabel(r"$f$")
    ax.set_ylim(-3, 3)
    ax.view_init(elev=25, azim=135)
    ax.legend(loc="upper left", fontsize=9)
    save(fig, "surface_3d.png")


def plot_subdifferential():
    """
    Графік субдиференціалу ∂f:
      (а) ∂f(x) у площині x: вектори-градієнти поза лінією та «віяла» на лінії негладкості;
      (б) множини ∂f(x) у просторі (g1, g2): точки поза лінією, відрізки на лінії;
      (в), (г) перерізи графіка багатозначного відображення ∂f уздовж прямих через x*.
    """
    fig, axs = plt.subplots(2, 2, figsize=(12, 10.5))

    # (а) площина x
    ax = axs[0, 0]
    contour_background(ax, (-4, 2), (-1.5, 3))
    for x1 in np.arange(-3.5, 2.01, 1.0):
        for x2 in np.arange(-1.0, 3.01, 1.0):
            x = np.array([x1, x2])
            if abs(kink(x)) < 0.3:
                continue
            g = subgradient(x) * 0.06
            ax.arrow(x1, x2, g[0], g[1], width=0.012, color="tab:blue", alpha=0.7,
                     length_includes_head=True)
    for x1 in [-3.0, -2.0, -1.0, 0.0, 1.0]:
        x = np.array([x1, (x1 + 4) / 2])
        p, q = subdifferential(x)
        for v in np.linspace(0, 1, 7):
            g = (p + v * (q - p)) * 0.06
            ax.plot([x[0], x[0] + g[0]], [x[1], x[1] + g[1]], color="tab:red", lw=1.3, alpha=0.8)
        ax.plot(*x, "o", color="tab:red", ms=4)
    ax.plot([], [], color="tab:blue", label=r"$\partial f(x) = \{\nabla f(x)\}$")
    ax.plot([], [], color="tab:red", label=r"$\partial f(x)$ — відрізок")
    ax.set_title("(а) субградієнти у точках площини (масштаб 0.06)")
    ax.legend(fontsize=8, loc="lower right")

    # (б) простір субградієнтів
    ax = axs[0, 1]
    ts = np.linspace(-3, 1, 9)
    cmap = plt.cm.plasma(np.linspace(0.05, 0.9, len(ts)))
    for c, t in zip(cmap, ts):
        x = np.array([t, (t + 4) / 2])
        p, q = subdifferential(x)
        ax.plot([p[0], q[0]], [p[1], q[1]], color=c, lw=4, solid_capstyle="round",
                label=rf"$x=({t:.1f},\,{(t + 4) / 2:.2f})$")
        # ліва і права границі: градієнти з обох боків лінії
        ax.plot(*p, "o", color=c, mec="k", ms=5)
        ax.plot(*q, "s", color=c, mec="k", ms=5)
    ax.plot(0, 0, "k+", ms=12, mew=2)
    ax.annotate(r"$0 \notin \partial f(x)$ на лінії", (0, 0), (1, -3), fontsize=9,
                arrowprops=dict(arrowstyle="->"))
    ax.set_xlabel(r"$g_1$")
    ax.set_ylabel(r"$g_2$")
    ax.set_title(r"(б) множини $\partial f(x)$ для точок лінії негладкості")
    ax.legend(fontsize=7, ncol=2, loc="upper left")

    # (в) переріз x2 = 1/3: перша компонента ∂f як функція x1 (стрибок у x1 = -10/3)
```

```
    ax = axs[1, 0]
    x2c = X_STAR[1]
    x1k = 2 * x2c - 4  # точка перетину з лінією негладкості
    for lo, hi in [(-6, x1k), (x1k, 2)]:
        t = np.linspace(lo, hi, 100)
        ax.plot(t, 4 * t + np.sign((lo + hi) / 2 - x1k), color="tab:blue", lw=2)
    ax.plot([x1k, x1k], [4 * x1k - 1, 4 * x1k + 1], color="tab:red", lw=4,
            label=rf"$\partial f$ у $x_1={x1k:.2f}$ — відрізок")
    ax.axhline(0, color="k", lw=0.8)
    ax.plot(X_STAR[0], 0, "*", color="gold", mec="k", ms=14, label=r"$0 \in \partial f(x^*)$")
    ax.set_xlabel(r"$x_1$  (при $x_2 = 1/3$)")
    ax.set_ylabel(r"$g_1 = 4x_1 + v$")
    ax.set_title(r"(в) графік $x_1 \mapsto \mathrm{pr}_1\,\partial f(x_1, 1/3)$")
    ax.legend(fontsize=8)

    # (г) переріз x1 = -1/4: друга компонента ∂f як функція x2 (стрибок у x2 = 15/8)
    ax = axs[1, 1]
    x1c = X_STAR[0]
    x2k = (x1c + 4) / 2
    for lo, hi in [(-1.5, x2k), (x2k, 3.5)]:
        t = np.linspace(lo, hi, 100)
        v = np.sign(x1c - 2 * (lo + hi) / 2 + 4)
        ax.plot(t, 6 * t - 2 * v, color="tab:blue", lw=2)
    ax.plot([x2k, x2k], [6 * x2k - 2, 6 * x2k + 2], color="tab:red", lw=4,
            label=rf"$\partial f$ у $x_2={x2k:.3f}$ — відрізок")
    ax.axhline(0, color="k", lw=0.8)
    ax.plot(X_STAR[1], 0, "*", color="gold", mec="k", ms=14, label=r"$0 \in \partial f(x^*)$")
    ax.set_xlabel(r"$x_2$  (при $x_1 = -1/4$)")
    ax.set_ylabel(r"$g_2 = 6x_2 - 2v$")
    ax.set_title(r"(г) графік $x_2 \mapsto \mathrm{pr}_2\,\partial f(-1/4, x_2)$")
    ax.legend(fontsize=8)

    fig.tight_layout()
    save(fig, "subdifferential.png")


def plot_trajectory(res, name, xlim=(-4.5, 3.5), ylim=(-2.5, 3.5)):
    """Траєкторія методу Поляка на лініях рівня + збільшений фрагмент біля x*."""
    fig, axs = plt.subplots(1, 2, figsize=(13, 5.5))
    for ax, (xl, yl) in zip(axs, [(xlim, ylim), ((-0.6, 0.1), (0.05, 0.6))]):
        contour_background(ax, xl, yl, levels=40)
        ax.plot(res["x"][:, 0], res["x"][:, 1], "o-", color=COLORS["Крок Поляка"], ms=4, lw=1.3,
                label="траєкторія")
        ax.plot(*res["x"][0], "s", color="tab:green", ms=9, label=r"$x^{(0)}$")
    axs[0].set_title("Траєкторія методу Поляка")
    axs[1].set_title("Збільшений фрагмент біля $x^*$")
    axs[0].legend(fontsize=8, loc="upper right")
    fig.tight_layout()
    save(fig, name)


def plot_convergence(res, name):
    """f(x^(k)) - f*, ||x^(k) - x*|| та крок alpha_k у логарифмічній шкалі."""
    fig, axs = plt.subplots(1, 2, figsize=(12, 4.5))
    k = np.arange(len(res["f"]))
    axs[0].semilogy(k, np.maximum(res["f"] - F_STAR, 1e-17), "o-", ms=3, label=r"$f(x^{(k)}) -
f^*$")
    axs[0].semilogy(k, np.linalg.norm(res["x"] - X_STAR, axis=1), "s-", ms=3,
                    label=r"$\|x^{(k)} - x^*\|$")
    axs[0].set_xlabel(r"ітерація $k$")
    axs[0].set_title("Збіжність методу Поляка")
    axs[0].legend()
    axs[1].semilogy(np.arange(len(res["alpha"])), res["alpha"], "o-", ms=3, color="tab:purple")
    # біля x* f — квадратична з гесіаном diag(4, 6); похибка лягає вздовж x1,
    # тому alpha_k -> 1/(2*4) = 1/8 (половина ньютонівського кроку)
    axs[1].axhline(1 / 8, color="gray", ls="--", label=r"$1/8$")
    axs[1].set_xlabel(r"ітерація $k$")
    axs[1].set_title(r"Довжина кроку $\alpha_k$")
    axs[1].legend()
    fig.tight_layout()
```

```
    save(fig, name)


# ---------------------------------------------------------------------------
# 4. Анімації
# ---------------------------------------------------------------------------
def _save_animation(anim, base, fps):
    anim.save(os.path.join(FIG_DIR, base + ".gif"), writer=PillowWriter(fps=fps), dpi=90)
    anim.save(os.path.join(FIG_DIR, base + ".mp4"), writer=FFMpegWriter(fps=fps, bitrate=2400),
dpi=130)
    print(f"  збережено {base}.gif / {base}.mp4")


def build_polyak_animation(res):
    """
    Анімація методу Поляка: (зліва) траєкторія і напрям поточного кроку,
    (посередині) «камера» у відносних координатах x - x*, вікно якої стискається
    разом з відстанню до розв'язку, (справа) збіжність f - f*.
    Повертає (fig, update, n_frames) — їх використовує і кінострічка для звіту.
    """
    xs, fs = res["x"], res["f"]
    n = len(xs)
    rel = xs - X_STAR
    gap = np.maximum(fs - F_STAR, 1e-17)
    fig, axs = plt.subplots(1, 3, figsize=(16, 5.6), gridspec_kw={"width_ratios": [1.15, 1, 1]})
    contour_background(axs[0], (-4.5, 3.5), (-2.5, 3.5), levels=30)
    (ln0,) = axs[0].plot([], [], "o-", color=COLORS["Крок Поляка"], ms=4, lw=1.4)
    (pt0,) = axs[0].plot([], [], "o", color="tab:orange", mec="k", ms=9, zorder=6)
    (ln1,) = axs[1].plot([], [], "o-", color=COLORS["Крок Поляка"], ms=4, lw=1.4, zorder=4)
    (pt1,) = axs[1].plot([], [], "o", color="tab:orange", mec="k", ms=9, zorder=6)
    axs[1].plot(0, 0, "*", color="gold", mec="k", ms=15, zorder=5)
    axs[1].set_aspect("equal")
    axs[1].set_xlabel(r"$x_1 - x_1^*$")
    axs[1].set_ylabel(r"$x_2 - x_2^*$")
    axs[1].ticklabel_format(style="sci", scilimits=(-2, 2))
    axs[2].set_xlim(0, n)
    axs[2].set_ylim(1e-16, gap.max() * 3)
    axs[2].set_yscale("log")
    axs[2].set_xlabel(r"ітерація $k$")
    axs[2].set_title(r"$f(x^{(k)}) - f^*$")
    (conv,) = axs[2].plot([], [], "o-", color=COLORS["Крок Поляка"], ms=4)
    axs[0].set_title("Траєкторія і напрям кроку")
    title = fig.suptitle("", fontsize=13)
    dyn = {"arrow": None, "cs": None, "kink": None}  # елементи, що перемальовуються щокадру

    def update(k):
        k = min(k, n - 1)
        ln0.set_data(xs[: k + 1, 0], xs[: k + 1, 1])
        pt0.set_data([xs[k, 0]], [xs[k, 1]])
        ln1.set_data(rel[: k + 1, 0], rel[: k + 1, 1])
        pt1.set_data([rel[k, 0]], [rel[k, 1]])
        for key in dyn:
            if dyn[key] is not None:
                dyn[key].remove()
                dyn[key] = None
        # «камера»: квадратне вікно півширини r навколо x*, лінії рівня — локальні
        r = max(1.6 * np.linalg.norm(rel[k]), 1e-8)
        axs[1].set_xlim(-r, r)
        axs[1].set_ylim(-r, r)
        D1, D2 = np.meshgrid(np.linspace(-r, r, 160), np.linspace(-r, r, 160))
        # f(x* + d) - f* без втрати точності (пряма різниця f - f* на масштабі 1e-8 — шум):
        # лінійні члени гладкої частини і модуля взаємно знищуються, бо 0 ∈ ∂f(x*)
        l_star = kink(X_STAR)
        Z = 2 * D1 ** 2 + 3 * D2 ** 2 + np.abs(l_star + D1 - 2 * D2) - l_star - (D1 - 2 * D2)
        dyn["cs"] = axs[1].contour(D1, D2, Z, levels=12, cmap="viridis",
                                   alpha=0.55, linewidths=0.8)
        t = np.array([-r, r]) + X_STAR[0]
        (dyn["kink"],) = axs[1].plot(t - X_STAR[0], (t + B_CONST) / 2 - X_STAR[1], "k--", lw=1.4)
        axs[1].set_title(f"Зум біля $x^*$ (півширина вікна {r:.1e})", pad=16)
        if k < n - 1:  # напрям кроку -alpha_k * g_k
```

```
            dyn["arrow"] = axs[0].annotate("", xs[k + 1], xs[k],
                                           arrowprops=dict(arrowstyle="->", color="tab:orange",
lw=2.2))
        conv.set_data(np.arange(k + 1), gap[: k + 1])
        title.set_text(rf"Метод Поляка, $k = {k}$:   $x^{{(k)}} = ({xs[k, 0]:.8f},\ {xs[k,
1]:.8f})$,   "
                       rf"$f(x^{{(k)}}) - f^* = {gap[k]:.2e}$")
        return [ln0, pt0, ln1, pt1, conv, title]

    fig.tight_layout(rect=(0, 0, 1, 0.94))
    return fig, update, n


def make_polyak_animation(res):
    fig, update, n = build_polyak_animation(res)
    frames = list(range(n)) + [n - 1] * 6  # кілька кадрів паузи наприкінці
    anim = FuncAnimation(fig, update, frames=frames, blit=False)
    _save_animation(anim, "polyak_anim", fps=3)
    plt.close(fig)

    # Кінострічка для pdf-звіту: кілька ключових кадрів тієї ж анімації
    fig, update, n = build_polyak_animation(res)
    picks = [0, 2, 8, n - 1]
    imgs = []
    for k in picks:
        update(k)
        fig.canvas.draw()
        imgs.append(np.asarray(fig.canvas.buffer_rgba()).copy())
    plt.close(fig)
    fig, axs = plt.subplots(len(picks), 1, figsize=(12, 3.9 * len(picks)))
    for ax, img in zip(axs, imgs):
        ax.imshow(img)
        ax.axis("off")
    fig.tight_layout(h_pad=0.3)
    save(fig, "polyak_frames.png")


def make_compare_animation(results, step=2):
    """Анімація одночасного руху всіх чотирьох методів з однієї стартової точки."""
    n = max(len(r["x"]) for r in results)
    fig, axs = plt.subplots(1, 2, figsize=(14, 5.6))
    contour_background(axs[0], (-4.5, 3.5), (-2.5, 3.5), levels=30)
    axs[1].set_yscale("log")
    axs[1].set_xlim(0, n)
    axs[1].set_ylim(1e-16, 100)
    axs[1].set_xlabel(r"ітерація $k$")
    axs[1].set_title(r"$f_{best}^{(k)} - f^*$")
    artists = []
    for r in results:
        c = COLORS[r["name"]]
        (ln,) = axs[0].plot([], [], "-", color=c, lw=1.3, label=r["name"])
        (pt,) = axs[0].plot([], [], "o", color=c, mec="k", ms=8)
        (cv,) = axs[1].plot([], [], "-", color=c, lw=2, label=r["name"])
        artists.append((r, ln, pt, cv))
    axs[0].legend(fontsize=8, loc="upper right")
    axs[1].legend(fontsize=8, loc="upper right")
    title = fig.suptitle("")

    def update(k):
        for r, ln, pt, cv in artists:
            m = min(k, len(r["x"]) - 1)
            ln.set_data(r["x"][: m + 1, 0], r["x"][: m + 1, 1])
            pt.set_data([r["x"][m, 0]], [r["x"][m, 1]])
            cv.set_data(np.arange(m + 1), np.maximum(r["f_best"][: m + 1] - F_STAR, 1e-17))
        title.set_text(f"Порівняння методів, ітерація k = {k}")
        return [a for _, *a in artists] + [title]

    fig.tight_layout()
    frames = list(range(0, n, step)) + [n - 1] * 8
    anim = FuncAnimation(fig, update, frames=frames, blit=False)
```

```
    _save_animation(anim, "methods_compare", fps=12)
    plt.close(fig)


# ---------------------------------------------------------------------------
# 5. Обчислювальні експерименти
# ---------------------------------------------------------------------------
def summary(res):
    """Підсумкові показники запуску."""
    return {
        "name": res["name"],
        "x0": res["x"][0].tolist(),
        "n_iter": res["n_iter"],
        "x_best": res["x_best"].tolist(),
        "f_best": float(res["f_best"][-1]),
        "gap": float(res["f_best"][-1] - F_STAR),
        "dist": float(np.linalg.norm(res["x_best"] - X_STAR)),
    }


def first_iterations(res, m=12):
    """Таблиця перших ітерацій: k, x1, x2, f, l(x), alpha_k, ||x - x*||."""
    rows = []
    for k in range(min(m, len(res["x"]))):
        x = res["x"][k]
        rows.append({
            "k": k, "x1": float(x[0]), "x2": float(x[1]), "f": float(res["f"][k]),
            "l": float(kink(x)),
            "alpha": float(res["alpha"][k]) if k < len(res["alpha"]) else None,
            "dist": float(np.linalg.norm(x - X_STAR)),
        })
    return rows


def crossings(res):
    """Кількість перетинів траєкторією лінії негладкості."""
    s = np.sign([kink(x) for x in res["x"]])
    return int(np.sum(s[1:] * s[:-1] <= 0))


def main():
    os.makedirs(FIG_DIR, exist_ok=True)
    out = {"x_star": X_STAR.tolist(), "f_star": F_STAR}

    print("=" * 78)
    print(f"Точний мінімум: x* = ({X_STAR[0]}, {X_STAR[1]:.8f}),  f* = 85/24 = {F_STAR:.10f}")
    print("=" * 78)

    # --- E1. Базовий запуск з x0, де траєкторія перетинає лінію негладкості ---
    x0 = [-4.0, 3.0]
    base = polyak_method(x0)
    out["e1"] = {"summary": summary(base), "iters": first_iterations(base, 12),
                 "last": first_iterations(base, len(base["x"]))[-3:],
                 "crossings": crossings(base)}
    print(f"\nE1. Метод Поляка, x0 = {x0}: {base['n_iter']} ітерацій, "
          f"f - f* = {base['f_best'][-1] - F_STAR:.2e}, ||x - x*|| = {summary(base)['dist']:.2e}")
    print(f"{'k':>3} {'x1':>12} {'x2':>12} {'f':>14} {'l(x)':>10} {'alpha':>10} {'||x-x*||':>10}")
    for r in out["e1"]["iters"]:
        a = f"{r['alpha']:.4f}" if r["alpha"] is not None else "-"
        print(f"{r['k']:>3} {r['x1']:>12.6f} {r['x2']:>12.6f} {r['f']:>14.10f} {r['l']:>10.4f} "
              f"{a:>10} {r['dist']:>10.2e}")

    # --- E2. Різні стартові точки ---
    starts = [[3.0, -2.0], [-4.0, 3.0], [10.0, 10.0], [-2.0, 1.0], [-5.0, -3.0], [0.0, 2.0]]
    e2 = []
    print("\nE2. Різні стартові точки:")
    for s in starts:
        r = polyak_method(s)
        row = summary(r) | {"l0": float(kink(np.array(s))), "crossings": crossings(r)}
        e2.append(row)
```

```
        print(f"  x0 = {s}: l(x0) = {row['l0']:+.1f}, ітерацій {row['n_iter']:>3}, "
              f"перетинів {row['crossings']}, ||x - x*|| = {row['dist']:.2e}")
    out["e2"] = e2

    # --- E3. Неточно відоме f*: f_target = f* + delta ---
    deltas = [1e-1, 1e-2, 1e-3, -1e-3, -1e-2, -1e-1]
    e3, e3_runs = [], []
    print("\nE3. Вплив похибки цільового рівня f_target = f* + delta (x0 = (-4, 3), 200
ітерацій):")
    for d in deltas:
        r = polyak_method(x0, f_target=F_STAR + d, max_iter=200)
        row = summary(r) | {"delta": d}
        e3.append(row)
        e3_runs.append((d, r))
        print(f"  delta = {d:+.0e}: ітерацій {row['n_iter']:>3}, f_best - f* = {row['gap']:.2e}, "
              f"||x_best - x*|| = {row['dist']:.2e}")
    tl = polyak_target_level(x0, max_iter=200)
    est = polyak_estimated_fstar(x0, max_iter=200)
    out["e3"] = {"rows": e3, "target_level": summary(tl), "estimated": summary(est)}
    print(f"  оцінка f* = f_best - 10/(10+k): f_best - f* = {est['f_best'][-1] - F_STAR:.2e}, "
          f"||x_best - x*|| = {summary(est)['dist']:.2e}")
    print(f"  динамічний рівень (f* невідоме): f_best - f* = {tl['f_best'][-1] - F_STAR:.2e}, "
          f"||x_best - x*|| = {summary(tl)['dist']:.2e}")

    # --- E4. Порівняння з іншими спробами (однаковий бюджет 200 ітерацій) ---
    others = [polyak_method(x0), diminishing_step_method(x0), constant_step_method(x0),
momentum_method(x0)]
    out["e4"] = []
    print("\nE4. Порівняння методів (x0 = (-4, 3), до 200 ітерацій):")
    print(f"  {'Метод':<26} | {'ітер.':>5} | {'f_best - f*':>11} | {'||x_best - x*||':>15}")
    for r in others:
        s = summary(r)
        # перша ітерація, на якій досягнуто f_best - f* < 1e-3
        hit = np.nonzero(r["f_best"] - F_STAR < 1e-3)[0]
        s["k_1e3"] = int(hit[0]) if hit.size else None
        out["e4"].append(s)
        print(f"  {s['name']:<26} | {s['n_iter']:>5} | {s['gap']:>11.2e} | {s['dist']:>15.2e}")

    with open(os.path.join(FIG_DIR, "results.json"), "w", encoding="utf-8") as fp:
        json.dump(out, fp, ensure_ascii=False, indent=1)

    # --- Рисунки ---
    print("\nРисунки:")
    plot_surface()
    plot_subdifferential()
    plot_trajectory(base, "polyak_trajectory.png")
    plot_convergence(base, "polyak_convergence.png")

    # E2: траєкторії з різних стартових точок
    fig, ax = plt.subplots(figsize=(8, 7))
    contour_background(ax, (-6, 11), (-4, 11), levels=30)
    for s, c in zip(starts, plt.cm.tab10(np.arange(len(starts)))):
        r = polyak_method(s)
        ax.plot(r["x"][:, 0], r["x"][:, 1], "o-", color=c, ms=3, lw=1.2,
                label=f"$x^{{(0)}}=({s[0]:g},\\,{s[1]:g})$, {r['n_iter']} ітер.")
    ax.legend(fontsize=8, loc="upper left")
    ax.set_title("E2: метод Поляка з різних стартових точок")
    save(fig, "e2_starts.png")

    # E3: похибка цільового рівня
    fig, ax = plt.subplots(figsize=(9, 4.8))
    for (d, r), c in zip(e3_runs, plt.cm.coolwarm(np.linspace(0, 1, len(e3_runs)))):
        ax.semilogy(np.maximum(r["f_best"] - F_STAR, 1e-17), color=c, lw=2, label=rf"$\delta = {d:
+g}$")
    ax.semilogy(np.maximum(est["f_best"] - F_STAR, 1e-17), ":", color="tab:brown", lw=2.5,
                label=r"$f_{best} - 10/(10+k)$")
    ax.semilogy(np.maximum(tl["f_best"] - F_STAR, 1e-17), "--", color="tab:gray", lw=2,
label="динамічний рівень")
    ax.semilogy(np.maximum(base["f_best"] - F_STAR, 1e-17), color="k", lw=2.5,
                label=r"$\delta = 0$ (точне $f^*$)")
```

```
    ax.set_xlabel(r"ітерація $k$")
    ax.set_ylabel(r"$f_{best}^{(k)} - f^*$")
    ax.set_title(r"E3: крок Поляка з цільовим рівнем $f^* + \delta$")
    ax.legend(fontsize=8, ncol=2)
    save(fig, "e3_target.png")

    # E4: порівняння методів
    fig, axs = plt.subplots(1, 2, figsize=(13, 5))
    contour_background(axs[0], (-4.5, 1), (-0.5, 3.5), levels=30)
    for r in others:
        c = COLORS[r["name"]]
        axs[0].plot(r["x"][:, 0], r["x"][:, 1], ".-", color=c, ms=3, lw=1, label=r["name"])
        axs[1].semilogy(np.maximum(r["f_best"] - F_STAR, 1e-17), color=c, lw=2, label=r["name"])
    axs[0].legend(fontsize=8, loc="upper right")
    axs[0].set_title("Траєкторії")
    axs[1].set_xlabel(r"ітерація $k$")
    axs[1].set_title(r"$f_{best}^{(k)} - f^*$")
    axs[1].legend(fontsize=8)
    fig.tight_layout()
    save(fig, "e4_compare.png")

    # --- Анімації ---
    print("\nАнімації:")
    make_polyak_animation(base)
    make_compare_animation(others)


if __name__ == "__main__":
    main()
```